

Servicio: gestion_protocolos.py

Archivo: `services/gestion_protocolos.py` · Documento técnico-pedagógico · Generado 2026-05-20 13:46

Construye el protocolo médico que le corresponde a cada paciente según el tipo de examen, si trabaja en mina y qué riesgos tiene. Define la lista de exámenes a realizar, el precio total y la ruta de áreas que debe recorrer.

Resumen general

El cambio más importante: antes el protocolo era solo un **texto que se imprimía y se perdía**. Ahora es una **estructura de datos rica** (diccionario con items, total, áreas, texto) que se guarda dentro de cada Cita y determina por qué áreas debe pasar el paciente. Además ahora se pueden persistir cotizaciones sueltas sin cita.

ANTES	AHORA
crear_protocolo(...) -> str (solo retornaba el texto)	construir_protocolo(...) -> dict {'items', 'total', 'areas', 'texto'}
El protocolo no tenía vínculo con áreas del flujo (las áreas eran iguales para todos)	items_a_areas(items) calcula la ruta de cada paciente según sus exámenes
Las cotizaciones sueltas se perdían al mostrarse en pantalla	cotizaciones = [] y registrar_cotizacion() las guardan con ID (COT-001, COT-002...)
Los tipos de examen estaban hardcoded como '1', '2', '3', '4'	TIPO_A_OPCION / OPCION_A_TIPO mapean Ingreso/Anual/Salida/Alto Riesgo a códigos

1. De devolver texto a devolver datos estructurados

Antes del cambio: El código original concatenaba todo en un único string con líneas separadas. Eso significaba que NO podías saber programáticamente cuánto era el total, qué áreas necesitaba el paciente, ni reusar la información para otra cosa.

Código original

```
def crear_protocolo(opcion, es_mina, riesgos_seleccionados):
    items_protocolo = []
    total = 0
    if opcion == "1":
        items_protocolo.extend([...])
    ...
    lines = [...]
    for descripcion, precio in items_protocolo:
        lines.append(f" - {descripcion:<35} S/ {precio:>6.2f}")
        total += precio
    ...
    return "\n".join(lines)  # <-- solo texto, sin estructura
```

Código actual

```
def construir_protocolo(opcion, es_mina, riesgos_seleccionados):
    items = _items_por_opcion(opcion, es_mina, riesgos_seleccionados)
    total = sum(p for _, p in items)
    areas = items_a_areas(items)
    # ... construir texto formateado ...
    return {
        "items": items,          # [(nombre_examen, precio), ...]
        "total": total,         # número
        "areas": areas,         # lista de áreas
        "texto": ...            # texto formateado
```

```

"areas": areas,          # lista ordenada de áreas a recorrer
"es_mina": es_mina,
"riesgos": list(riesgos_seleccionados or []),
"texto": "\n".join(lines),
}

# Mantenemos la antigua para no romper código existente:
def crear_protocolo(opcion, es_mina, riesgos_seleccionados):
    return construir_protocolo(opcion, es_mina, riesgos_seleccionados)["texto"]

```

Para principiantes: Un **diccionario** en Python (escrito con `{}`) es una estructura que guarda pares **clave: valor**. Es como una ficha donde puedes acceder a cada dato por su nombre: `protocolo["total"]`, `protocolo["areas"]`. Mucho más flexible que un solo string.

2. El mapa examen → área del flujo

Cuando un paciente tiene un examen de *Audiometría*, sabemos que debe pasar por el área **Audiometría**. Pero cuando tiene *Evaluación Laboratorio*, debe ir al área **Laboratorio**. Este diccionario hace esa traducción de manera centralizada.

```

EXAMEN_A_AREA = {
    "Médica Ocupacional": "Medicina Ocupacional",
    "Evaluación Visual": "Triage",
    "Audiométrica": "Audiometría",
    "Audiometría": "Audiometría",
    "Evaluación Laboratorio": "Laboratorio",
    "Radiográfica de Tórax": "Rayos X",
    "Espirometría": "Laboratorio",
    "Enfermedades Ocupacionales": "Medicina Ocupacional",
    "Medicina General": "Medicina Ocupacional",
}

ORDEN_AREAS = ["Triage", "Laboratorio", "Audiometría",
               "Rayos X", "Medicina Ocupacional"]

```

Para principiantes: El `ORDEN_AREAS` define el flujo lógico: siempre Triage primero, después Laboratorio, etc. Si un paciente no necesita una de esas áreas, simplemente la salta. Pero el orden de visita es consistente para todos.

3. Cotizaciones que se guardan

Antes, cuando un usuario generaba una cotización sin tener aún un paciente registrado, el resultado se mostraba en pantalla y se perdía al cerrar la ventana. Ahora cada cotización tiene su propio ID y se queda en memoria durante la sesión.

```

cotizaciones = [] # lista global de cotizaciones generadas en la sesión

def registrar_cotizacion(opcion, es_mina, riesgos_seleccionados):
    proto = construir_protocolo(opcion, es_mina, riesgos_seleccionados)
    proto["id"] = f"COT-{len(cotizaciones) + 1:03d}" # COT-001, COT-002...
    proto["timestamp"] = datetime.now()
    proto["tipo"] = OPCION_A_TIPO.get(opcion, opcion)
    proto["cita_asignada"] = None # se puede asignar después a una cita
    cotizaciones.append(proto)
    return proto

```

Para principiantes: `f'COT-{n:03d}'` es una **f-string** con formato: el `:03d` rellena el número con ceros a la izquierda hasta tener 3 dígitos. Así obtenemos IDs ordenados visualmente (COT-001, COT-002, COT-010, COT-100).